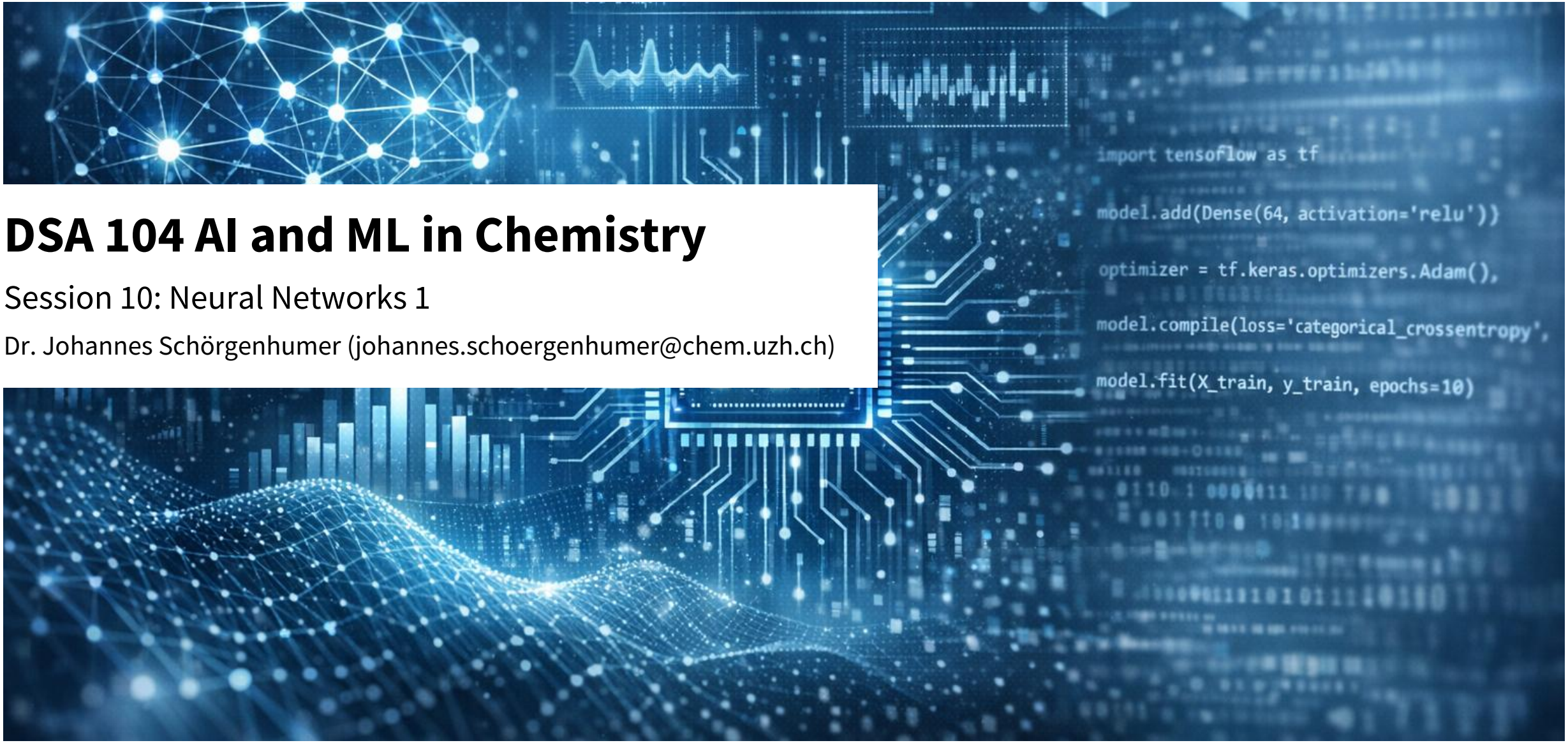
Universität Zürich[UZH]

Department of Chemistry

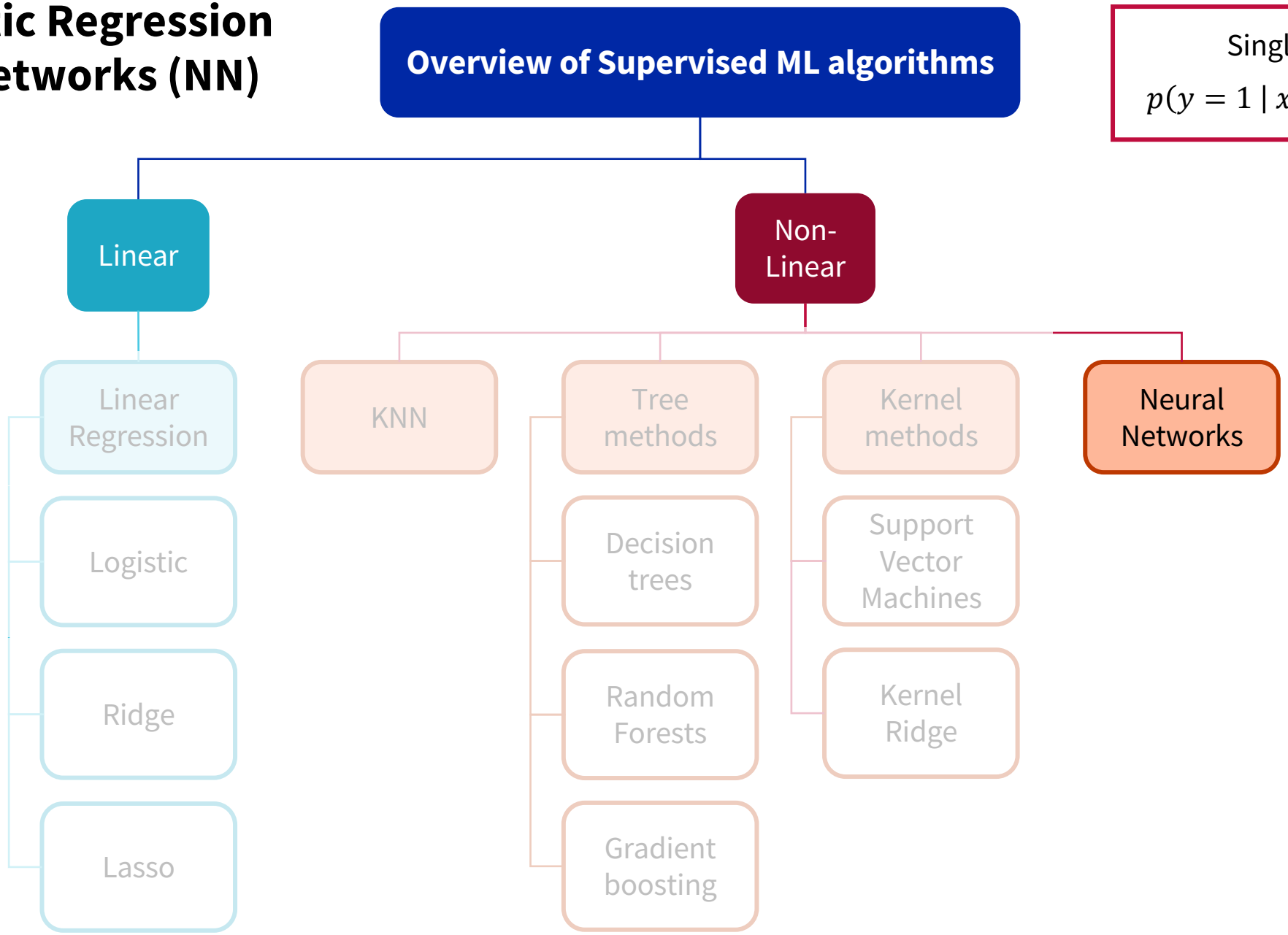


DSA 104 AI and ML in Chemistry

Session 10: Neural Networks 1

Dr. Johannes Schörgenhumer (johannes.schoergenhumer@chem.uzh.ch)

From Logistic Regression to Neural Networks (NN)

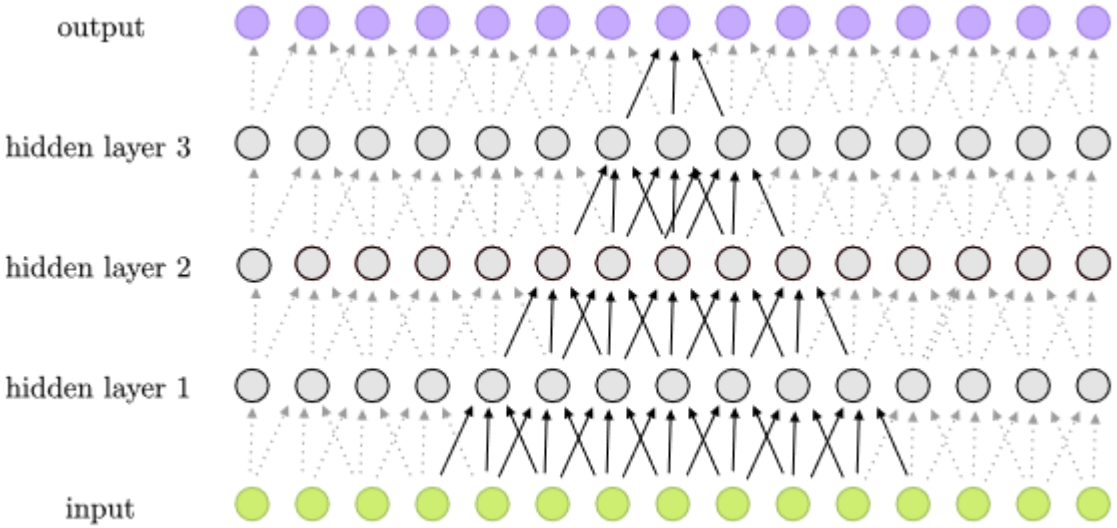


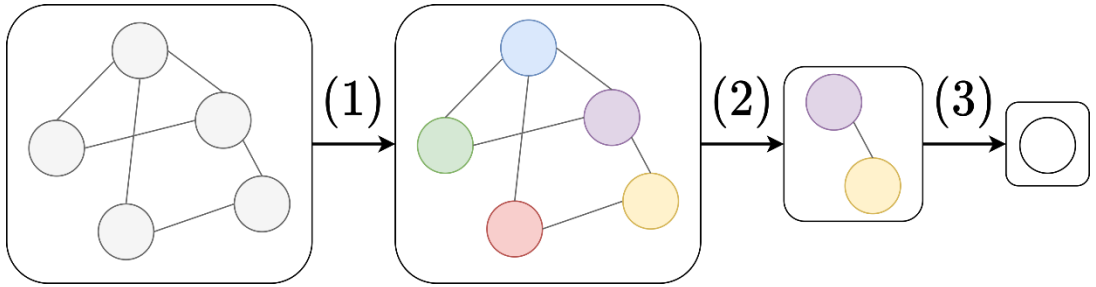
Single neuron:

$$p(y = 1 \mid x) = \sigma(w^{\mathrm{T}} x + b)$$

Types of Neural Networks

- **FNN/MLP (Feedforward NN)**
- **CNN (Convolutional NN)**
- RNN (Recurrent NN)
- Transformers
- Encoders
- GAN (Generative Adversarial N)
- **GNN (Graph NN)**
- RL networks (Reinforcement Learning N)
- Diffusion models





CNN illustration by Vicente Oyanedel M. - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=95810663
Basic building blocks for GNN by NickDiCicco - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=119932650

Recap: Intro to Neural Networks (NNs)

Neurons / Perceptrons:

— Basic computational unit of a neural network

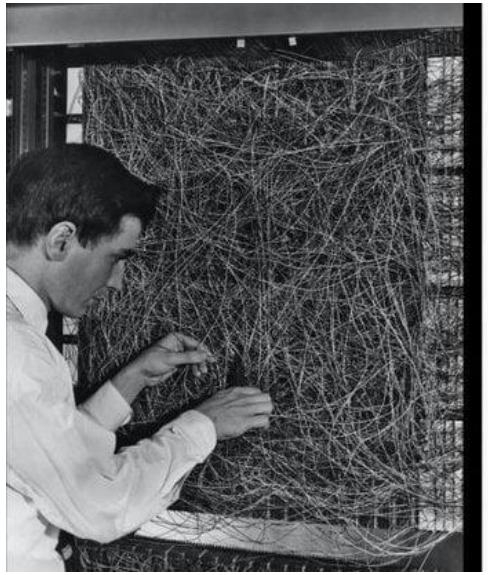
— Takes a vector of inputs, computes a weighted sum plus bias ($z = w^{\mathrm{T}}x + b$)

— Applies activation function ($a = \Phi(z)$)

— If activation potential passed: Output

— Weights and bias learned from data during training

— Concept developed 1957 by Frank Rosenblatt:



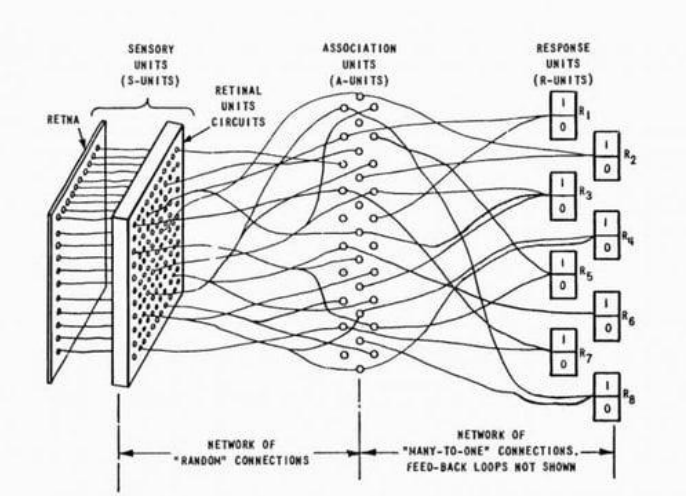
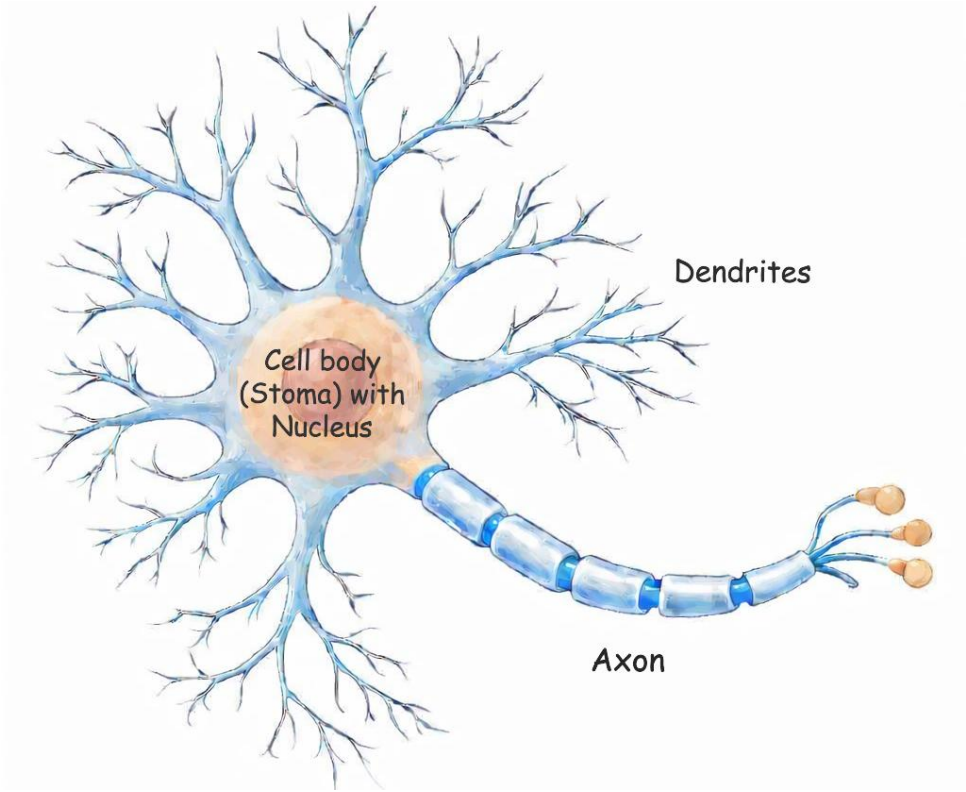
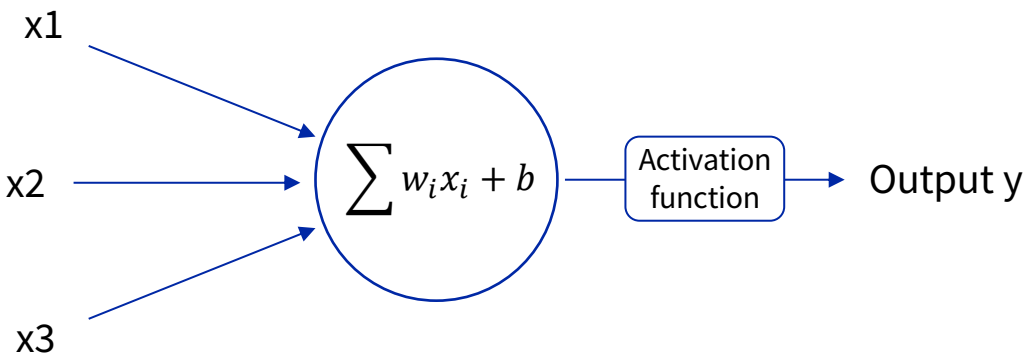


Figure 1 ORGANIZATION OF THE MARK I PERCEPTRON





F. Rosenblatt, *Psychol. Rev.* **1958**, *65*, 386-408, e.g. available in: J.A. Anderson, E. Rosenfeld, "Neurocomputing, Volume 1: Foundations of Research", MIT Press, **1988**; https://doi.org/10.7551/mitpress/4943.003.0010.
Photo from: https://data-science-blog.com/blog/2020/07/16/a-brief-historyof-neural-nets-everything-you-should-know-before-learning-lstm/

Recap neurons:

— Neuron = linear model + activation

— E.g. $y = f(w_1 x_1 + w_2 x_2 + b)$

— Weights *w* determine importance of features

— Bias *b* shifts the boundary

— Single-layer neurons limited:

  — $x \rightarrow output$

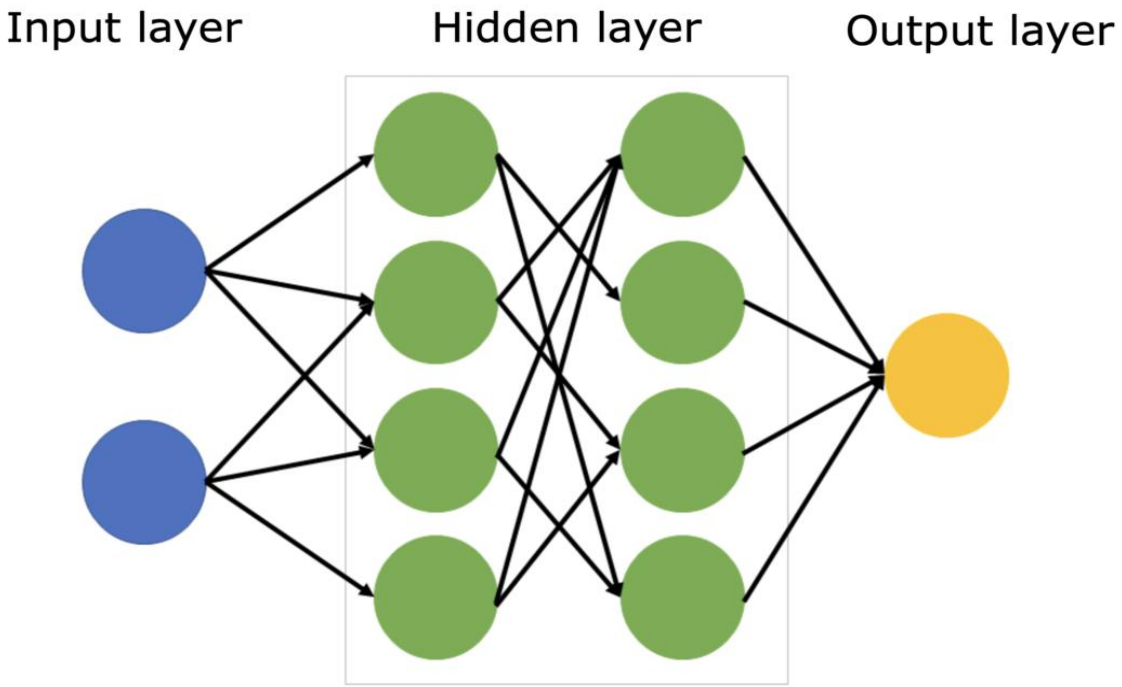
  — Only linear patterns can be learned

— Hence, **multilayer networks** (a.k.a. multilayer perceptron (MLP = FNN)):

  — $x \rightarrow hidden \rightarrow hidden \rightarrow output$

  — Hidden layer(s) between input and output layer, each computing: $h = f(Wx + b)$

  — **Non-linear patterns can be learned**



Artificial neural networks

Image by seKiChun, CC BY-SA 4.0, https://www.sciencelearn.org.nz/images/5156-neural-network-diagram

Multilayer Networks

Input layer:

Accepts raw data (numbers, pixels, vectors)

Hidden layer(s):

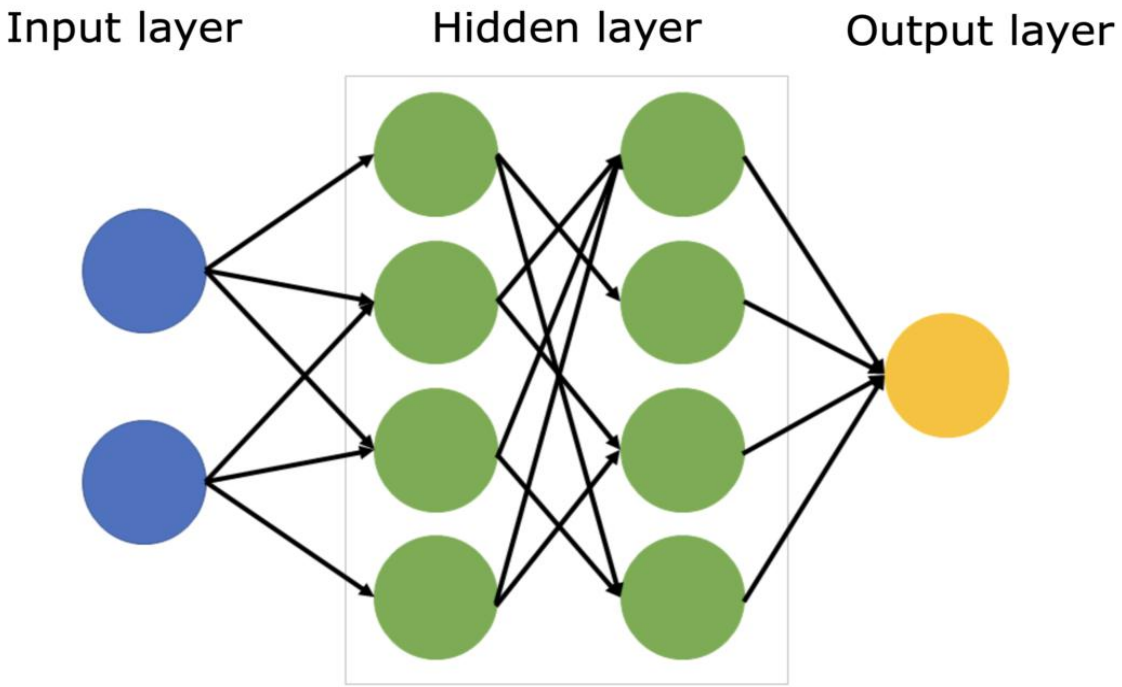
Transform representations using nonlinearities

Output layer:

Produces predictions (classification, regression)

More layers = deeper network (Deep learning!)

More neurons per layer = wider network



Artificial neural networks

MLPs shine when data is:
Structured (tabular, small vectors)
Nonlinear but not spatial or sequence-based
Moderate-dimensional

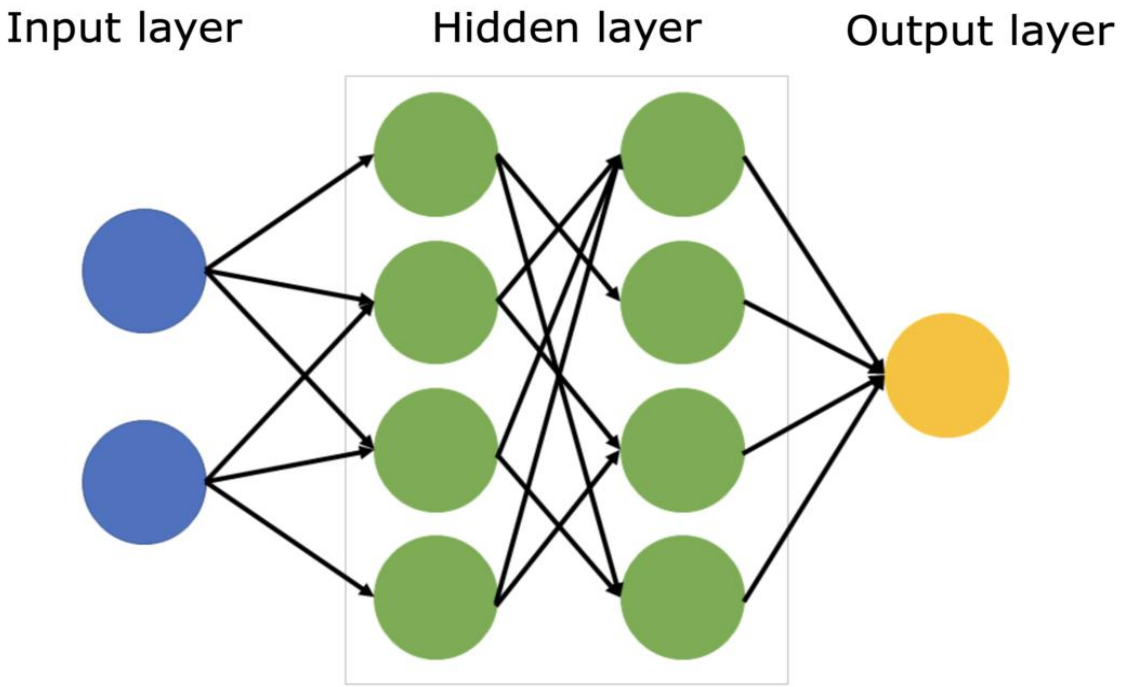
Image by seKiChun, CC BY-SA 4.0, https://www.sciencelearn.org.nz/images/5156-neural-network-diagram

Forward propagation / forward pass – how the network predicts:

Computing output!

— The information passes **forward** through the network in several steps:

1. Applying linear layers
2. Compute weighted sum
3. Apply activation
4. Pass prediction/outcome to next layer
5. Repeat for each layer



Artificial neural networks

Image by seKiChun, CC BY-SA 4.0, https://www.sciencelearn.org.nz/images/5156-neural-network-diagram

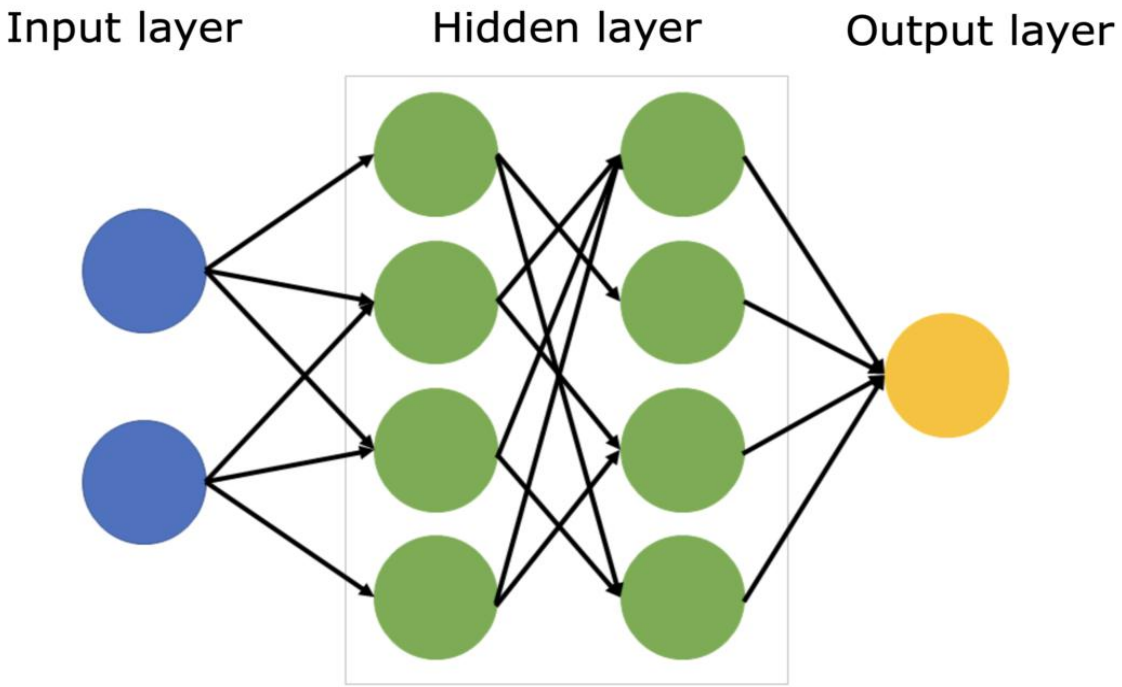
Backpropagation – how the network learns:

Computing gradients!

Training a multilayer network involves:

1. **Forward pass**: Compute predictions through all layers
2. **Compute loss**: Compare predictions to true labels
3. **Backward pass** (backpropagation): Compute gradients of all weights via the chain rule
4. **Update weights**: Using an optimiser (e.g., SGD, Adam)

This loop repeats many times (=**epochs**) over the dataset



Artificial neural networks

Image by seKiChun, CC BY-SA 4.0, https://www.sciencelearn.org.nz/images/5156-neural-network-diagram

Learning as Optimisation:

Find weights that minimise loss!

— Update rule: $w = w - \nabla L(w)$

Some common **gradient descent methods**:

— Adam (default in PyTorch):

  — runs average of gradients and squared gradients

  — Converges fast

— AdamW:

  — adds decoupled weight decay

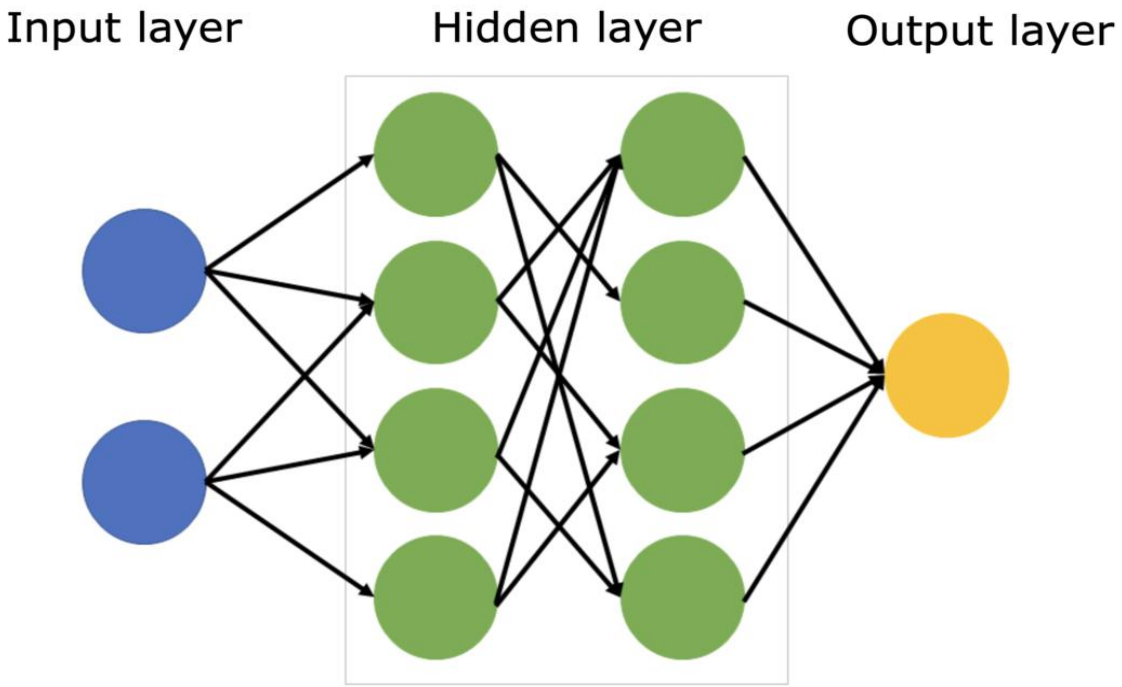
  — THE optimiser for modern DL (Transformers!)

— SGD: High accuracy but slow

— Adagrad:

  — Increases learning rate for infrequent features

  — Well suited for sparse data (bad in deep networks)



Artificial neural networks

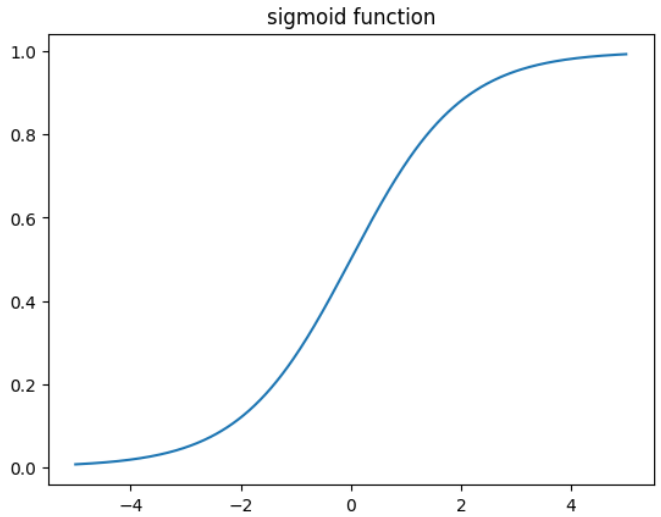
Activation functions

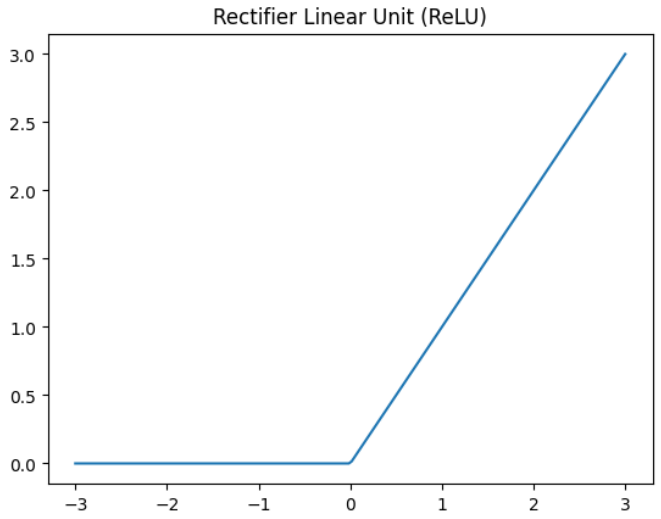
What happens if no activation functions are used?

Introduce **nonlinearity**!

Some common activation functions:

- **Sigmoid**: for binary output, Range: (0, 1) $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Tanh**: centered around 0, Range: (-1, 1) $\tanh x$
- **ReLU**: Simple, efficient, widely used in deep nets (avoids vanishing gradients) $f(x) = \max(0, x) = \frac{x+|x|}{2} = \begin{cases} x \text{ if } x > 0 \\ 0 \text{ if } x \leq 0 \end{cases}$
- **Softmax**: smooth approximation of ReLU $f(x) = \ln(1 + e^x)$





Building guide – robust baseline:

Don’t over-engineer from the start: start small and only add depth/width when needed

Solid starting points:

- Feedforward, fully connected NN:
  - **Hidden layers**: start with **2-4**
  - **Width**: **2x to 8x** the input dimension

- Other models (CNNs, RNNs, Transformers):
  - Use standard backbones / pretrained models

Building guide – robust baseline:

Activation functions:

Tasks	Good Default	Notes
Most tasks	ReLU	Simple, fast, widely used
Very deep nets	GELU	Used in modern transformers
Output for regression	No activation	Linear output
Output for binary classification	Sigmoid	With BCEWithLogitsLoss
Output for multi-class classification	Softmax	With CrossEntropyLoss

Loss functions:

Task	Recommended Loss	Why
Binary classification	BCEWithLogitsLoss (or BCELoss)	Combines sigmoid + BCE
Multi-class classification	CrossEntropyLoss	Includes softmax internally
Regression	MSELoss, L1Loss	MSE smoother, L1 robust to outliers
Imbalanced classes	CrossEntropyLoss(weight=...)	Add class weighting

Building guide – robust baseline:

- **Batch size** :
  - How many datapoints loaded in each training iteration
  - Noisy data and regularisation: smaller batch sizes
  - Depending on memory, e.g. 32 to 64 for GPU computing

- **Learning rate**:
  - Most models: **0.001 to 0.01** as good starting point

- **Regularisation**:
  - **Dropout** for fully-connected nets: start with 0.1 to 0.3
  - **Weight decay**: use default (1e-4) with **AdamW** (deep nets!)
  - **Batch normalisation**: use between layers for stability

PyTorch Workshop

1) Pytorch demo:

— Load data into tensors

— Build NN – basic syntax

2) Optimise the NN for the diabetes dataset!

Investigate in pairs / small groups:

— Width and depth of the NN

— Epochs

— Learning_rate

— Loss functions

— Optimisers (e.g. compare trining time)

— Compare to the results of the linear regressions

What happens if you simply run the train the model again without reinitialising the model (rerunning the cell where the model is first called)?